

Individual Assessment Coversheet

To be attached to the front of the assessment.

Campus: Tyger Valley

Faculty: Information Technology

Module Code: ITSEA2-12

Group: 1

Lecturer's Name: Sandura Shumba

Student Full Name: Keenan Paiva Batista

Student Number: 4845350

Indicate	Yes	No
Plagiarism report attached		

Declaration:

I declare that this assessment is my own original work except for source material explicitly acknowledged. I also declare that this assessment or any other of my original work related to it has not been previously, or is not being simultaneously, submitted for this or any other course. I am aware of the AI policy and acknowledge that I have not used any AI technology to generate or manipulate data, other than as permitted by the assessment instructions. I also declare that I am aware of the Institution's policy and regulations on honesty in academic work as set out in the Conditions of Enrolment, and of the disciplinary guidelines applicable to breaches of such policy and regulations.

Signature 	Date 20 March 2025
---	------------------------------

Lecturer's Comments:

--

Marks Awarded:	%
-----------------------	----------

Signature	Date
------------------	-------------

Table of Contents

Section A

Question 1.....Page 1-2

Question 2.....Page 3-5

Question 3..... Page 6

Question 4..... Page 7

Bibliography.....Page 8

Section A

Question 1

1.1) Software engineering refers to a systematic approach to the production of software where engineering principles and techniques are utilised. Software engineering principles can be used by businesses to define what should be done at each stage of creating software, including designing, development, testing and maintaining software. The principles of software engineering can also be altered to accommodate the specific requirements of any large-scale systems like PIMS, which can increase its effectiveness when addressing the unique issues and maintenance tasks associated with the specific large-scale system. Moreover, software engineering can solve general issues related to large-scale systems such as software development costs and employee productivity, because the best approaches and techniques to achieve these requirements can be planned for by using software engineering (Mills, H.D., 1999: 294).

1.2) A systematic approach refers to creating a plan and utilising it in order to achieve specified objectives. Creating and using a plan to address the issues associated with PIMS is vital to ensure that these various challenges can be solved in an efficient and effective manner. A systematic approach enables project members to effectively co-ordinate solutions for each complex challenge identified in PIMS. Furthermore, this can also enable the project team to complete their task within the specified budget and deadline due to the increase in efficiency and productivity (Rönkkö, K., Dittrich, Y. & Randall, D., 2005: 437).

Additionally, a systematic approach can be further benefit PIMS by reducing potential confusion when implementing changes to the system internationally. Documentation that is created from the systematic approach for PIMS enables all physical locations that use PIMS to understand how the changes will be implemented to the system, which greatly reduces confusion allowing for PIMS to be updated without any issues caused by staff.

1.3) Each software system will require software engineering techniques that can either be different or similar to each other. Hospital management and e-commerce systems in most situations will require an agile model to be used during software development and maintenance to ensure that changes can be made easily when requirements are altered or added, such as new healthcare, business compliance requirements and new security mechanisms to address recent security threats. The weather monitoring application usually will require a plan-driven model since any requirement changes will be less frequent than software like the hospital management and e-commerce system.

Additionally, the e-commerce system can differ from the hospital management system by utilising refactoring through extreme programming while the hospital management system can use prototyping. Refactoring enables an e-commerce system to greatly improve the efficiency of its operations such as user management and users purchasing products. Prototyping enables executable models of the hospital management system to be created and tested stakeholders for feedback. The weather monitoring application can use interviews to ensure that the requirements from stakeholders are fully understood and implemented.

There are various factors can determine the challenges that each system can face such as risks associated with the development of the software, the difficulty of gaining the requirements from

users and the overall size of the software (Gao, Y. & Feng, X.Z., 2012: 4-5). The hospital management and e-commerce system can utilise incremental delivery to ensure that changes can be easily made to protect against the risk of sensitive data being accessed by unauthorised users and abide to new user requirements. The weather monitoring system can utilise the waterfall model due to the system being a general system that won't require constant changes, and it won't need additional components to protect data as there isn't any sensitive data associated with it.

1.4) Healthcare systems have the responsibility of ensuring that patient data remains accurate and safeguarded against malicious attackers. Patient data is invaluable for patients and caregivers, because this data can be analysed and used to reduce the chances of inappropriate care and improve beneficial care (Snyder, K.D. & Paulson, P., 2002: 6). If patient data is not protected in the software system against tampering and stealing it can endanger patients as they could receive incorrect treatment and medication as well as make the business face legal repercussions for not being compliant with regulations regarding patient data privacy. Patient data can be protected by implementing security mechanisms like authentication to protect the privacy of the patients and prevent malicious tampering.

Moreover, ensuring system reliability is another responsibility when creating and maintaining a healthcare system. The system should be reliable both mechanically and functionally where a failure in function and a reduction in the system's performance due to component degradation should be avoided (Son, Y.K., 2011: 1717). If the healthcare system experiences constant performance issues and shutdowns, it can greatly reduce the efficiency of care to patients as medical staff won't have continual access to important data and resources. This can be prevented by ensuring that constant maintenance and improvements are made to the system and its individual components with methods like refactoring.

Additionally, healthcare systems must promote user inclusivity by ensuring that the system is widely accessible and accommodates by all users with various conditions. Failing to promote user inclusivity in the design of the system can increase the difficulty for certain patients to effectively utilise the system, which reduces the overall quality of care for patients. User inclusivity can be implemented by conducting surveys to receive feedback from the patients that specifies the issues that they have with the healthcare system, which enables for these problems to then be solved.

Question 2

2.1) Modernizing a legacy system such as PIMS can be quite risky to perform depending on how essential the system is to the operations of the business. Modernizing a legacy system will require a set of objectives to be defined and a plan that indicates how new enhancements should be implemented that enables users to use them (Francisca, O.O. & Okwudili, A.S., 2011: 245-246). The software process model that can solve the issues associated with PIMS is the waterfall model due to its ability to effectively design large-scale, international systems.

The waterfall model is a plan-driven model that provides sequential steps to designing software systems and is beneficial to updating each aspect of PIMS, where design flaws are identified and corrected before the software is developed or updated. Moreover, the documentation and planning involved with the waterfall model makes it more effective for quality control in PIMS when compared to other software process models (Alshamrani, A. & Bahattab, A., 2015: 1). PIMS requires an increased level of quality control to determine how bottlenecks and outdated interfaces can be improved, identify and solve security vulnerabilities as well as implement new features that enables scalability. In addition, the disadvantage of the waterfall model not being able to effectively apply new changes when new requirements are found, will not impact the modernization of PIMS since most requirements are already defined and any potential requirements is minimised due to PIMS being a generic instead of a customized system.

The other software process models, incremental development as well as integration and configuration, are not as effective at solving the issues associated with PIMS when compared to the waterfall model. Incremental development combines aspects of the waterfall model with the ability to develop and update software incrementally (Alshamrani, A. & Bahattab, A., 2015: 2). This ability of incremental development can't be fully taken advantage of due to PIMS being a general instead of a customized system. Additionally, the flexibility of the incremental model can create confusion due to PIMS having to be updated at several locations internationally, where a more structured plan like the waterfall method is appropriate. Furthermore, integration and configuration refers to reusing and configuring other software to create or update a system to meet user requirements. Integration and configuration is the least useful software process model for PIMS, because of the risks associated with adding additional software components to a legacy system while ensuring that it can interact with legacy components without creating new issues or destroying the system.

2.2) A security policy framework should be developed that defines security procedures to ensure that PIMS remains secure from potential attacks. After the security policy framework is established for PIMS, it should be used in conjunction with the waterfall model, called the policy-driven waterfall model, where security and scalability principles are implemented throughout the whole software development process (Hussain, S., Anwaar, H., Sultan, K., Mahmud, U., Farooqui, S., Karamat, T. & Ibrahima, K.T., 2024: 6).

Security mechanisms can be developed during software development where the security policy framework is used as a guideline, the security mechanisms should also be tested during software testing to ensure that vulnerabilities are minimized, and it doesn't negatively impact the system. The security mechanisms should also be alterable to ensure that new attacks that are discovered can be protected against. The same methods used to ensure that the system is secure after the software is fully developed can also be applied to implement methods that enables for scalability in the system.

2.3) If a plan-driven model like the waterfall model is used to modernize PIMS, the system can be designed to apply features that can accommodate requirement changes during the maintenance phase of the process. Preparing for requirement changes to occur in the future is beneficial for a plan-driven model, because it can make up for the lack of flexibility when the system is designed by enabling it to be altered after implementation. This is especially useful for PIMS as patient and medical staff users can request for new features to be added or changed as well as becoming compliant with new healthcare requirements.

Furthermore, an agile model like incremental development, as well as integration and configuration, offer more options to easily implement changes to the system than a plan-driven model. The agile model used can be altered to add system prototyping and incremental delivery during the overall software process, which enables users like doctors and patients to provide feedback to updates and request new requirements for PIMS.

2.4) Various requirement changes can occur during the lifecycle of PIMS that can have a negative impact if the system fails to accommodate it. Failing to be compliant to new healthcare regulations can result in Paradise Healthcare facing legal repercussions. The severity of the legal repercussions depends on what healthcare regulation was violated such as ensuring that sensitive patient data is protected against unauthorised users. An additional negative impact of failing to meet regulations include patients losing trust in the business.

Moreover, failing to meet new security requirements to minimise discovered vulnerabilities and prevent new attacks against the system can also make PIMS untrustworthy to patients and uncompliant with certain healthcare policies if attackers are able to steal sensitive patient data. Security attacks can also lead to the system itself being damaged requiring time and funds to properly recover from it.

2.5) Software process improvement (SPI) refers to analysing existing software processes and determining how these processes can be improved upon to minimise aspects such as poor productivity and software quality (Iqbal, J., Jibran, H., Al-Shamayleh, A., Abbas, F., Akhunzada, A., Alharthi, S.Z. & Gani, A., 2023: 2). SPI can be practiced for a system like PIMS to address current challenges and ensure that it won't become an issue in the future. Improving software processes through SPI can ensure that existing performance, scalability and security issues in PIMS are solved in an efficient manner, where the system is designed to enable developers to easily solve it in the future and new project members can understand the system through effective documentation.

However, various factors can negatively impact the quality of new software processes for PIMS. These factors can occur during the analysis stage where project members can have differing opinions on what is problematic with the software processes and actual problems could be missed or ignored (Kautz, K., Henrik, W.H. & Thaysen, K., 2001: 6). The factor can be minimised by determining how the problem identified can be solved and how it will benefit the process in developing the system, if solving the problem doesn't greatly benefit the system then new problems should be identified and analysed. Additionally, all meetings and discussions related improving software processes could not be properly communicated to all relevant individuals where software process improvements could be done incorrectly (Kautz, K., Henrik, W.H. & Thaysen, K., 2001: 13). This can be minimised by establishing a shared communication channel with all relevant individuals and make these meetings mandatory for all relevant individuals.

2.6) A benefit of utilising an agile approach is that it enables developers to easily implement changes to systems based on new requirements when compared to a plan-driven approach that doesn't offer the same flexibility for each stage. For PIMS, an agile approach enables developers

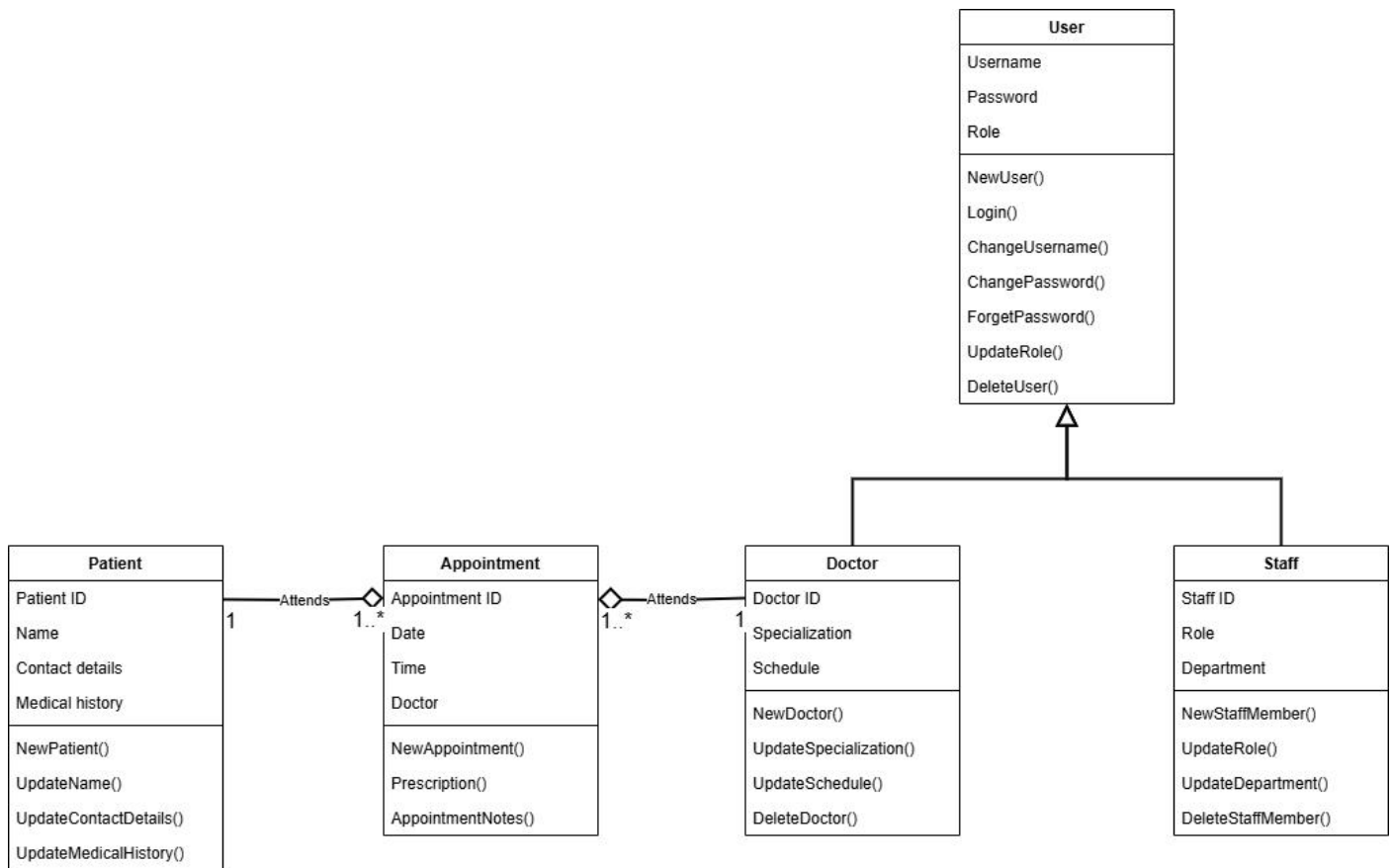
to implement suggestions from the medical staff and patients as well as solve vulnerabilities that were identified, while a plan-driven approach can't apply changes as easily and efficiently.

However, the plan-driven approach ensures that all aspects of a system update and development can be planned for in a manner where it can be applied to several locations and documented effectively for future reference, while an agile approach doesn't focus as much on documentation and confusion can be produced when attempting to implement it at several locations. A plan-driven approach is beneficial for PIMS as it's used in multiple locations, due to Paradise Healthcare being international. The documentation can be used for future reference since PIMS is a legacy system and new project members will need to understand how it functions. An agile approach can't provide these benefits as effectively as a plan-driven approach.

The best-fit methodology for PIMS would be a plan-driven approach, because it can effectively solve the issues that are specific to PIMS where an agile approach is more appropriate for other systems like PIMS where the issues are less complex.

Question 3

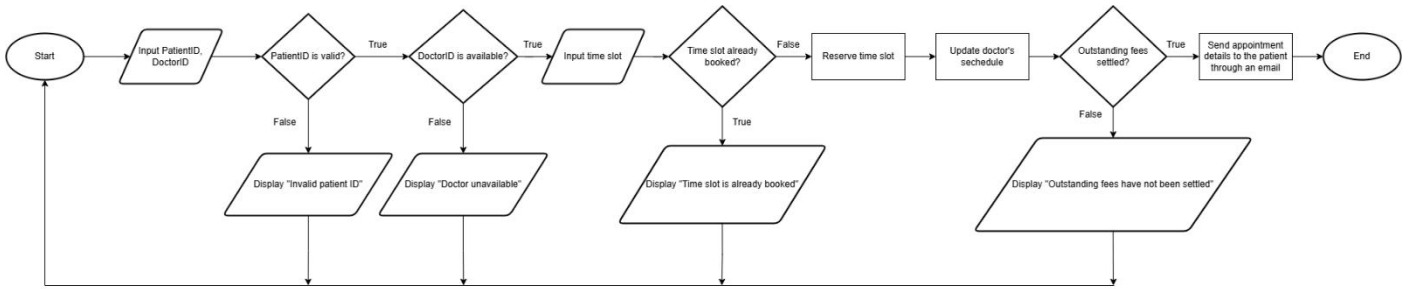
3.1)



3.2) Class diagrams are the best way to model the relationships in PIMS due to the ability to generalise classes, categorise the type of relationships between classes and indicate the attributes and functions of classes. Identifying each entity in PIMS and allocating appropriate attributes and functions enables project members to recognise the critical components that make up PIMS and how it interacts with other aspects of the system through the relationships that are indicated in the diagram.

This allows project members to identify areas where challenges can occur, such as performance issues between the patient and doctors when applying for an appointment and implement changes in a manner that doesn't negatively impact other components of PIMS. Furthermore, the ability to generalise classes also enables project members to reduce the complexity of the system by grouping classes with similar characteristics to improve project members' understanding of it.

Question 4



When documenting the process of how a system will complete certain tasks through a flowchart, it enables individuals to gain a better understanding of the system processes more efficiently than other documentation methods. Flowcharts are effective at clarifying the system processes, because individuals are able to understand and analyse information quicker with visual components than written paragraphs. Flowcharts are very useful when information needs to be conveyed in an effective manner to new project members and stakeholders.

Moreover, flowcharts can also reduce the number of potential errors that can occur in a system due to how effectively flowcharts are able to communicate system processes. The flowchart can act as a guide for the project team when developing the system, which enables them to predict potential issues that can occur during the system processes as well as create easily create step-by-step plans of how the system can be effectively tested for any errors that were not detected.

Bibliography

- Francisca, O.O. & Okwudili, A.S. 2011, "A SOFTWARE REVERSE ENGINEERING METHODOLOGY FOR LEGACY MODERNIZATION", *International Journal of Advances in Engineering & Technology*, vol. 1, no. 4, pp. 244-248. [Online] Available at ProQuest [Accessed: 16 March 2025]
- Alshamrani, A. & Bahattab, A. 2015, "A Comparison Between Three SDLC Models Waterfall Model, Spiral Model, and Incremental/Iterative Model", *International Journal of Computer Science Issues (IJCSI)*, vol. 12, no. 1, pp. 106-111. [Online] Available at ProQuest [Accessed: 16 March 2025]
- Hussain, S., Anwaar, H., Sultan, K., Mahmud, U., Farooqui, S., Karamat, T. & Ibrahima, K.T. 2024, "Mitigating Software Vulnerabilities through Secure Software Development with a Policy-Driven Waterfall Model", *Journal of Engineering*, vol. 2024. [Online] Available at ProQuest [Accessed: 16 March 2025]
- Iqbal, J., Jibran, H., Al-Shamayleh, A., Abbas, F., Akhunzada, A., Alharthi, S.Z. & Gani, A. 2023, "Comparative study of SPI success factors in global and in-house environment for large-scale software companies", *PeerJ Computer Science*, . [Online] Available at ProQuest [Accessed: 18 March 2025]
- Kautz, K., Henrik, W.H. & Thaysen, K. 2001, "Understanding and Changing Software Organisations: An Exploration of Four Perspectives on Software Process Improvement", *Scandinavian Journal of Information Systems*, vol. 13, no. 1, pp. 4. [Online] Available at ProQuest [Accessed: 18 March 2025]
- Mills, H.D. 1999, "The management of software engineering, part I: Principles of software engineering", *IBM Systems Journal*, vol. 38, no. 2, pp. 289-295. [Online] Available at ProQuest [Accessed: 19 March 2025]
- Rönkkö, K., Dittrich, Y. & Randall, D. 2005, "When Plans do not Work Out: How Plans are Used in Software Development Projects", *Computer Supported Cooperative Work*, vol. 14, no. 5, pp. 433-468. [Online] Available at ProQuest [Accessed: 19 March 2025]
- Gao, Y. & Feng, X.Z. 2012, "Factors to be Considered When to Design Software Development Plan", *Advanced Engineering Forum*, vol. 6-7, pp. 3. [Online] Available at ProQuest [Accessed: 19 March 2025]
- Snyder, K.D. & Paulson, P. 2002, "Healthcare information systems: Analysis of healthcare software", *Hospital topics*, vol. 80, no. 4, pp. 5-12. [Online] Available at ProQuest [Accessed: 20 March 2025]
- Son, Y.K. 2011, "Reliability prediction of engineering systems with competing failure modes due to component degradation", *Journal of Mechanical Science and Technology*, vol. 25, no. 7, pp. 1717-1725. [Online] Available at ProQuest [Accessed: 20 March 2025]